

CI/CD with Jenkins + GenAI

Day 1: Cloud Foundations & DevOps Mindset

Your hands-on journey from cloud basics to a fully running EC2 environment — built for working IT professionals stepping into the DevOps world.



Day 1 Agenda

01

Cloud Basics

What the cloud is, why it matters, and how it works

02

AWS EC2 Hands-On

Launch a real server in the cloud

03

DevOps Fundamentals

The mindset, lifecycle, and CI/CD pipeline

04

Environment Setup

Java, Git, and SSH — getting your tools ready

05

AI Integration

Using GenAI to learn faster and debug smarter

What Is Cloud Computing?



Cloud computing means **using IT resources over the internet** — servers, storage, databases, networking — without owning any physical hardware.

→ On-demand access

Get compute power whenever you need it

→ Pay-as-you-go

Only pay for what you actually use

→ Globally accessible

Access from anywhere, at any time

Traditional vs. Cloud Infrastructure

Traditional IT

- Buy physical servers upfront
- Weeks to provision new capacity
- Fixed capacity — over or under-provisioned
- You maintain hardware & software

Cloud Infrastructure

- Rent virtual resources instantly
- Spin up servers in minutes
- Scale up or down on demand
- Provider handles the physical layer

 Think of it this way: Traditional IT is buying a car. Cloud is using Uber — same destination, far less overhead.

ANALOGY

Own a House vs. Rent an Apartment

Own a House = On-Premise

You buy land, build it, maintain it — full control, huge upfront cost, and you're stuck with it even if you move.

Rent an Apartment = Cloud

You pay monthly, move when you want, and the landlord handles repairs. Flexible, fast, and scalable.



Key Benefits of Cloud Computing



Scalability

Instantly grow or shrink resources to match demand — no over-provisioning



Cost Efficiency

Replace capital expenses with predictable operational costs



Speed

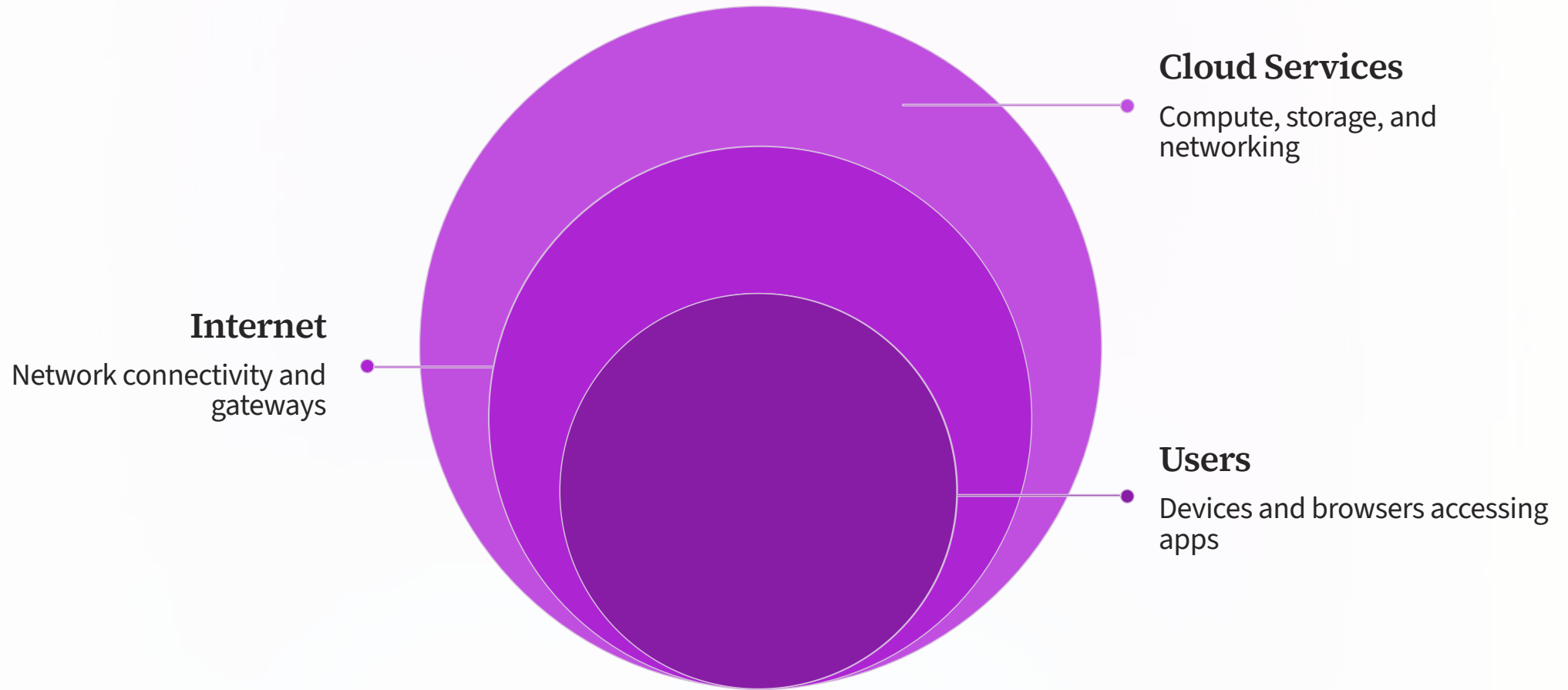
Deploy infrastructure in minutes instead of weeks



Reliability

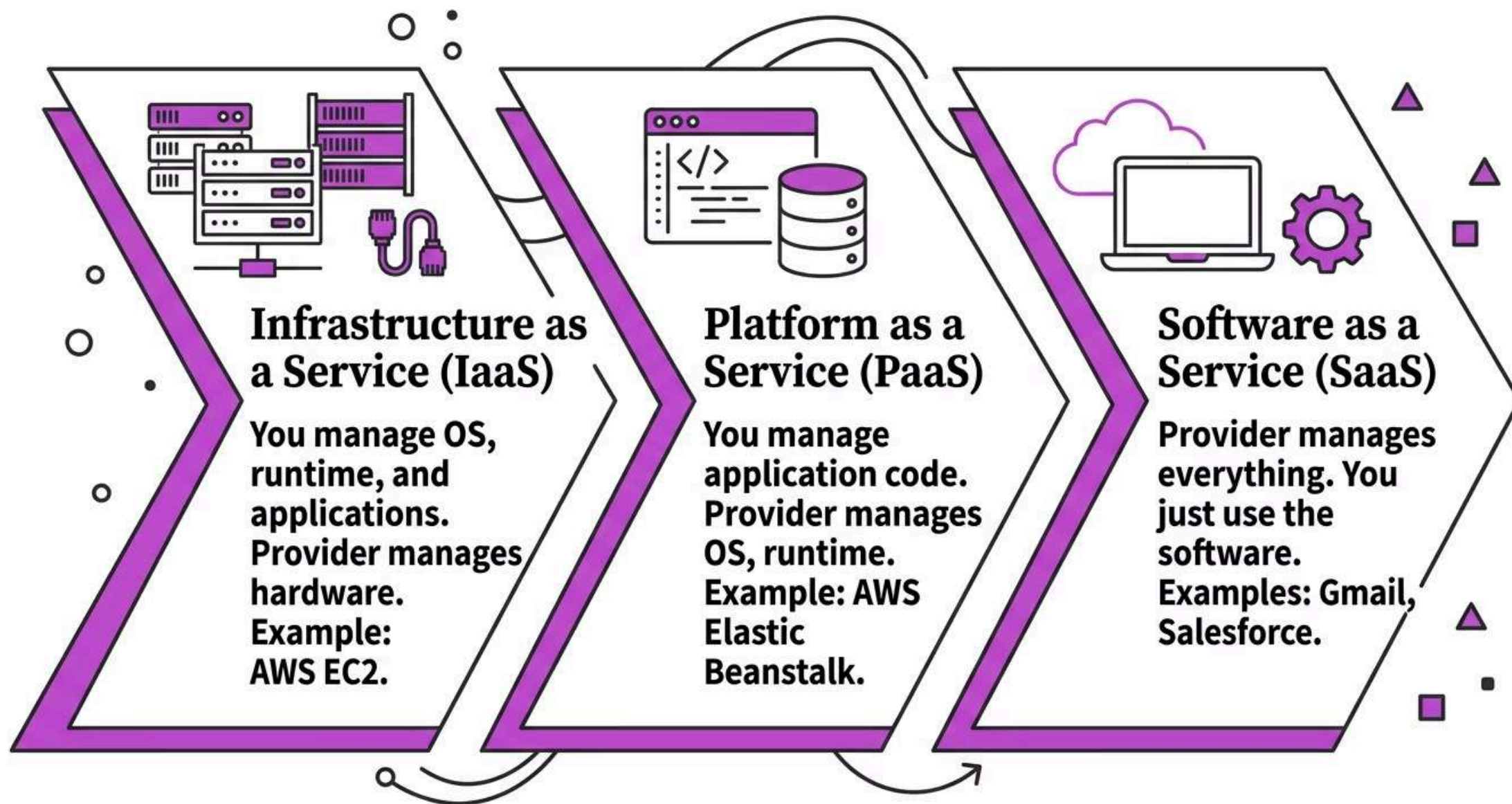
Built-in redundancy and global availability zones keep you online

Cloud Architecture at a Glance



Every cloud interaction flows through this stack — your app lives in the middle layers while the physical hardware is completely invisible to you.

The 3 Cloud Service Models



IaaS — Infrastructure as a Service



You get the raw infrastructure — virtual machines, storage, and networking — and you manage everything above the hardware.

❏ Best example: **AWS EC2** — you launch a virtual machine and configure it yourself.

You Control

OS, middleware, runtime, applications, and data

Provider Controls

Physical servers, networking, and storage hardware

PaaS — Platform as a Service



You focus on writing your application. The provider manages the OS, runtime, and infrastructure beneath it.

📄 Best example: **AWS Elastic Beanstalk** — upload your code, and AWS handles the rest.

You Control

Your application code and data — that's it

Provider Controls

OS, runtime, middleware, servers, and storage

SAAS

SaaS — Software as a Service



You just use the software. No installation, no servers, no maintenance — the provider handles absolutely everything.

- Access via a web browser or app
- Subscriptions replace licenses
- Updates happen automatically



Examples you already use: **Gmail, Salesforce, Slack, Microsoft 365**

IaaS vs. PaaS vs. SaaS — At a Glance

Feature	IaaS	PaaS	SaaS
You manage	OS, Apps, Data	Apps, Data	Nothing
Provider manages	Hardware	Hardware + OS	Everything
Flexibility	High	Medium	Low
Best for	DevOps, custom infra	App developers	End users
AWS Example	EC2	Elastic Beanstalk	Amazon WorkMail



ANALOGY

The Housing Analogy for Service Models



IaaS = Apartment

You get empty walls and plumbing. You furnish it, decorate it, maintain it your way.



PaaS = Serviced Apartment

Furniture included, cleaning done. You just bring your clothes and live there.



SaaS = Hotel

Room service, made bed, everything provided. Just show up and enjoy.

Cloud Deployment Models



Public Cloud

Shared infrastructure owned by a provider (AWS, Azure, GCP) — accessible to anyone



Private Cloud

Dedicated infrastructure for one organization — hosted on-premises or by a provider



Hybrid Cloud

Combination of public and private — workloads move between them as needed

Public Cloud — Open, Scalable, Shared



Public cloud is the most widely used model. Resources are hosted and managed by a provider and shared across many customers — with full isolation between them.

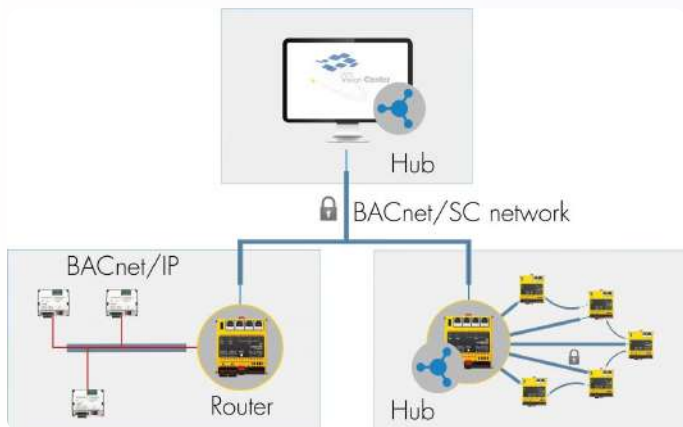
- Lowest upfront cost
- Infinite scalability
- Managed by AWS, Azure, or GCP

✔ Best for: Startups, development environments, web applications

Private Cloud — Dedicated, Controlled, Secure

A private cloud is used exclusively by a single organization. It offers maximum control and security — critical for industries with strict compliance requirements.

- Full control over hardware and data
- Meets strict regulatory standards
- Higher cost, more IT responsibility



i Real-world example: **Banks and hospitals** running their own on-premise clouds to protect sensitive data.

Hybrid Cloud — Best of Both Worlds



Hybrid cloud lets organizations keep sensitive workloads on-premises while bursting into the public cloud for scale or speed.

- Sensitive data stays private
- Use public cloud for peak demand
- Gradual migration path to cloud

📘 Real-world example: A **retail enterprise** processes payments on-premise but runs its e-commerce app on AWS.

When to Use Which Model?



Public Cloud

Startups, web apps, dev/test environments, unpredictable workloads



Private Cloud

Banking, healthcare, government — high security, compliance-driven workloads



Hybrid Cloud

Enterprises migrating gradually, or needing burst capacity during peak periods

Why Cloud Is the Foundation for DevOps

Fast Provisioning

Spin up a server in 2 minutes, not 2 weeks

Elastic Scalability

Match infrastructure to traffic — automatically

Automation-Ready

APIs and scripts replace manual ticketing

Cost Control

Pay only during active build/test cycles

Manual vs. Cloud-Enabled DevOps

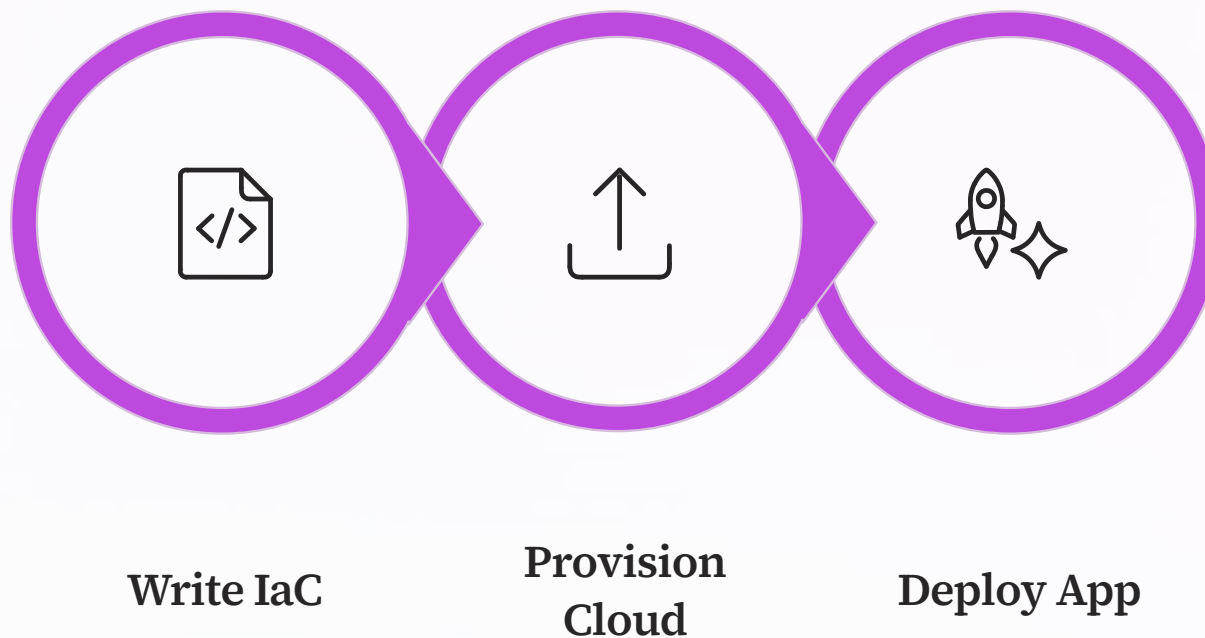
Manual (Traditional)

- Submit a ticket to get a server
- Wait 1–2 weeks for provisioning
- Manually install OS and dependencies
- Releases happen monthly — if you're lucky

Cloud-Enabled DevOps

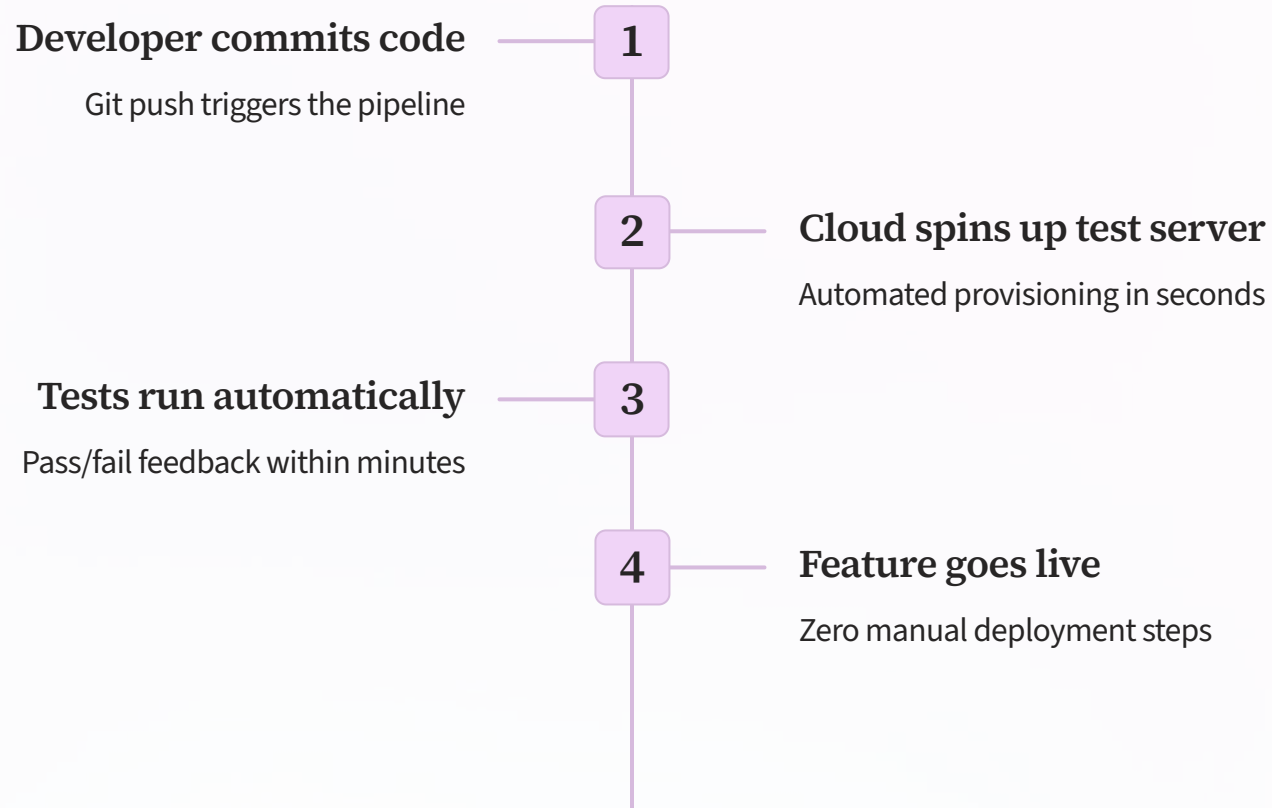
- Run a script — server ready in 2 minutes
- Dependencies installed automatically
- Code merged, tested, and deployed — same day
- Multiple releases per day, safely

Automation: The Cloud Superpower



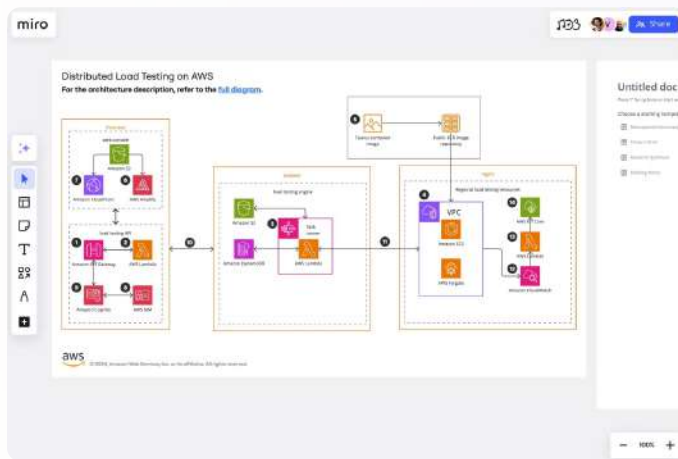
Infrastructure as Code (IaC) means your entire environment is version-controlled and reproducible — no more "it works on my machine" excuses.

Real-World Scenario: Launching a Feature Fast



This pipeline — powered by cloud infrastructure — is exactly what you'll build in this training.

What Is AWS?



Amazon Web Services (AWS) is the world's largest cloud platform, offering over 200 services — from computing and storage to AI and machine learning.

→ **Market leader**

~33% of global cloud market share

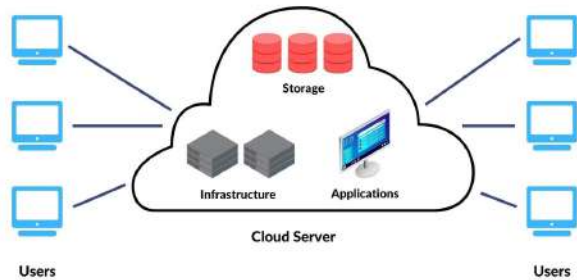
→ **Used everywhere**

Netflix, Airbnb, NASA, Samsung —
all run on AWS

→ **Free Tier available**

Practice for free — perfect for this
training

What Is EC2?



EC2 (Elastic Compute Cloud) is AWS's virtual server service. Think of it as renting a computer in Amazon's data center — you choose the size, OS, and configuration.

📌 Analogy: EC2 is like renting a laptop from Amazon — you get full control without buying the hardware.

Elastic

Resize your server anytime

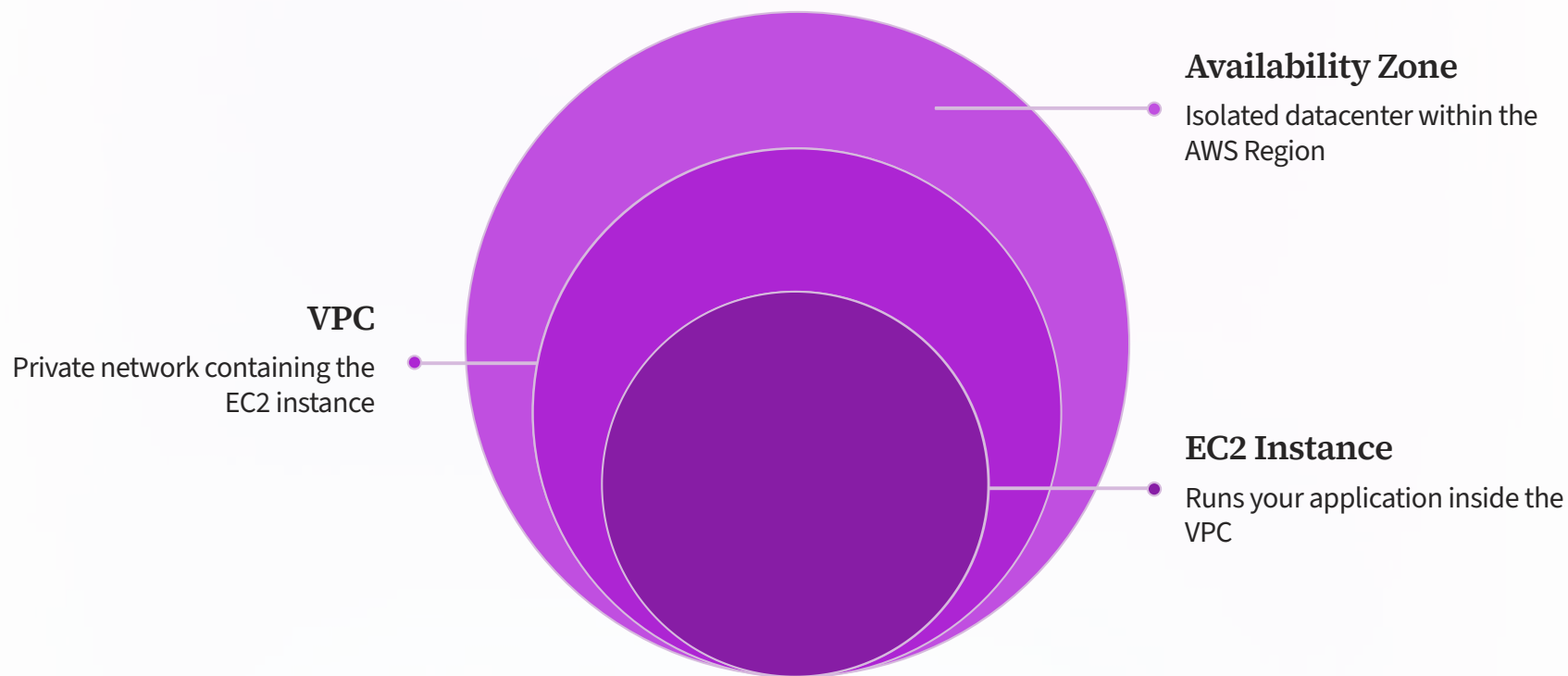
Compute

CPU + RAM for your workloads

Cloud

Hosted and managed by AWS

EC2 Architecture Overview



Your EC2 instance lives inside a VPC (your private network in AWS), which sits inside an Availability Zone — providing physical redundancy and isolation.

Key EC2 Components You Must Know



Instance

Your virtual machine — choose type (t2.micro for free tier), OS (Amazon Linux, Ubuntu), and storage size



Key Pair

Your SSH credentials — a **.pem file** you download once and use to securely log into your instance



Security Group

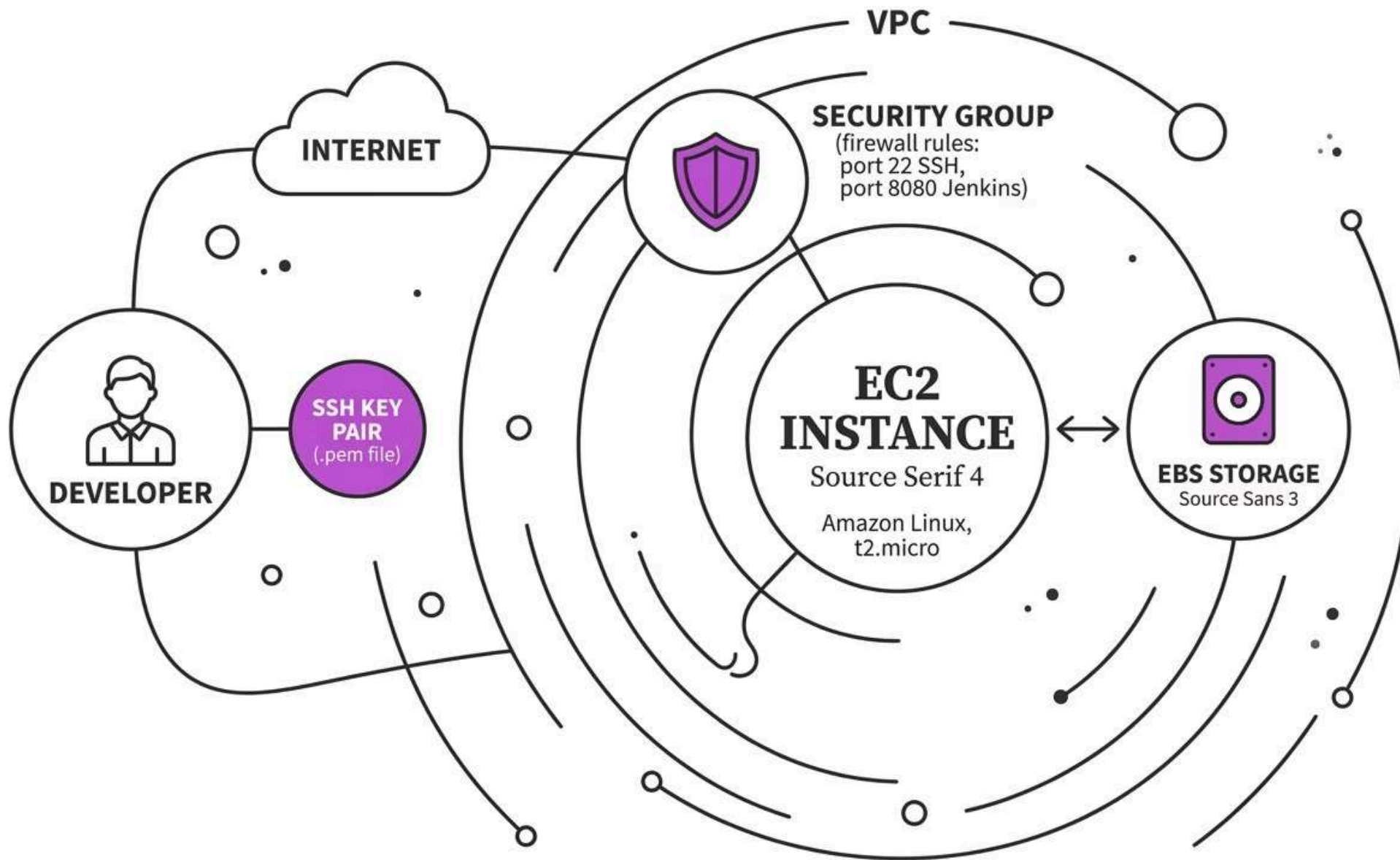
A virtual firewall — controls which ports are open (e.g., port 22 for SSH, port 8080 for Jenkins)



Public IP / Elastic IP

The address you use to connect — Public IP changes on restart; Elastic IP stays fixed

EC2 Setup Diagram



Instance Types — Picking the Right Size

Instance Type	vCPU	RAM	Best For
t2.micro	1	1 GB	Free tier, learning, light workloads
t2.medium	2	4 GB	Jenkins master, small apps
t3.large	2	8 GB	Production builds, heavier pipelines
m5.xlarge	4	16 GB	High-traffic enterprise workloads



For today's lab, we'll use **t2.micro** — it's free under the AWS Free Tier and perfect for learning.

What Is DevOps?



DevOps is a culture and set of practices that bridges the gap between software development (Dev) and IT operations (Ops) to deliver software faster and more reliably.

→ Break down silos

Dev and Ops work together, not in isolation

→ Automate everything

Build, test, and deploy with code — not clicks

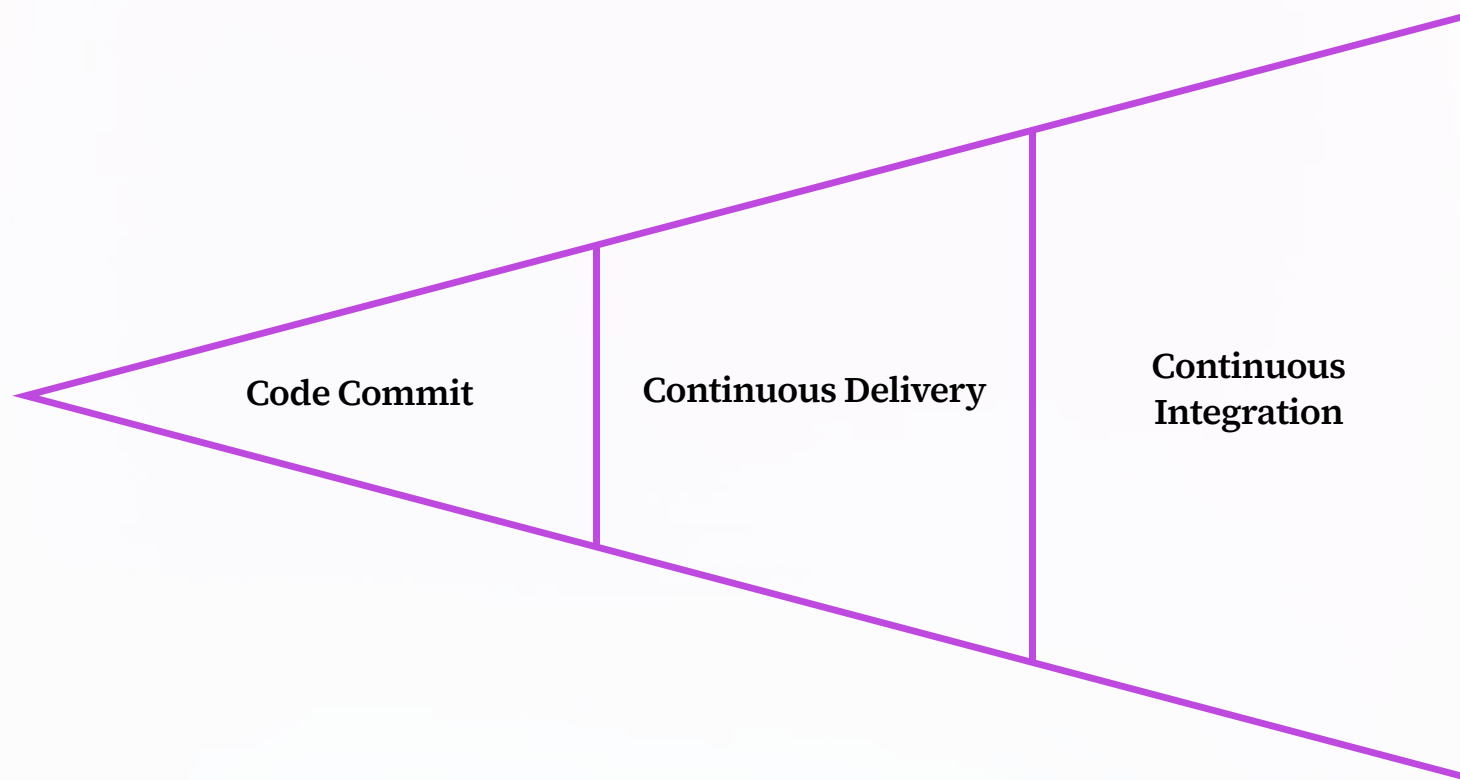
→ Deliver continuously

Ship small, ship often, fail safely

The DevOps Lifecycle



What Is CI/CD?



CI/CD is the backbone of modern software delivery — it replaces manual, error-prone releases with a fast, automated, and repeatable pipeline that runs every time code is pushed.

Traditional vs. Modern Pipelines

🐌 Traditional Pipeline

- Code written in silos for weeks
- Handed off to a QA team manually
- Big-bang releases every quarter
- Rollbacks are painful and slow

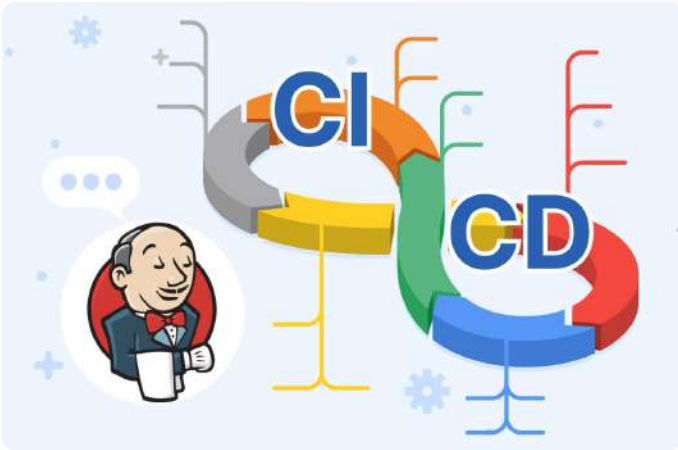
🚀 Modern CI/CD Pipeline

- Small commits merged daily
- Tests run automatically on every push
- Deploy multiple times per day
- Instant rollback with a single command

✅ Netflix deploys code **thousands of times per day** using exactly this approach.



Why Jenkins?



Jenkins is the world's most popular open-source automation server — the engine that powers CI/CD pipelines for millions of teams.

- **Free and open-source**
No license cost — just install and run
- **1800+ plugins**
Integrates with Git, Docker, AWS, Slack, and more
- **Pipeline as Code**
Define your entire pipeline in a `Jenkinsfile`



 GENAI INTEGRATION

How GenAI Accelerates Your Cloud Learning

Explain Concepts

Ask AI to explain IaaS or VPCs in plain English — with analogies

Generate Commands

Get the exact SSH or AWS CLI commands you need instantly

Debug Errors

Paste an error message — get a step-by-step fix in seconds

Write Configs

Generate Jenkinsfiles, shell scripts, and YAML automatically

Effective ChatGPT Prompts for Cloud & DevOps

Understanding Concepts

"Explain the difference between IaaS, PaaS, and SaaS using a real-world analogy. I'm a beginner."

Getting Commands

"Give me the exact command to SSH into an AWS EC2 Amazon Linux instance using a .pem file from Windows."

Debugging Help

"I'm getting 'Permission denied (publickey)' when connecting via SSH to EC2. What are the likely causes and fixes?"

Generating Configs

"Write a Jenkinsfile for a simple CI pipeline that runs a Maven build and unit tests on every Git push."

AI for Troubleshooting EC2 & SSH Issues



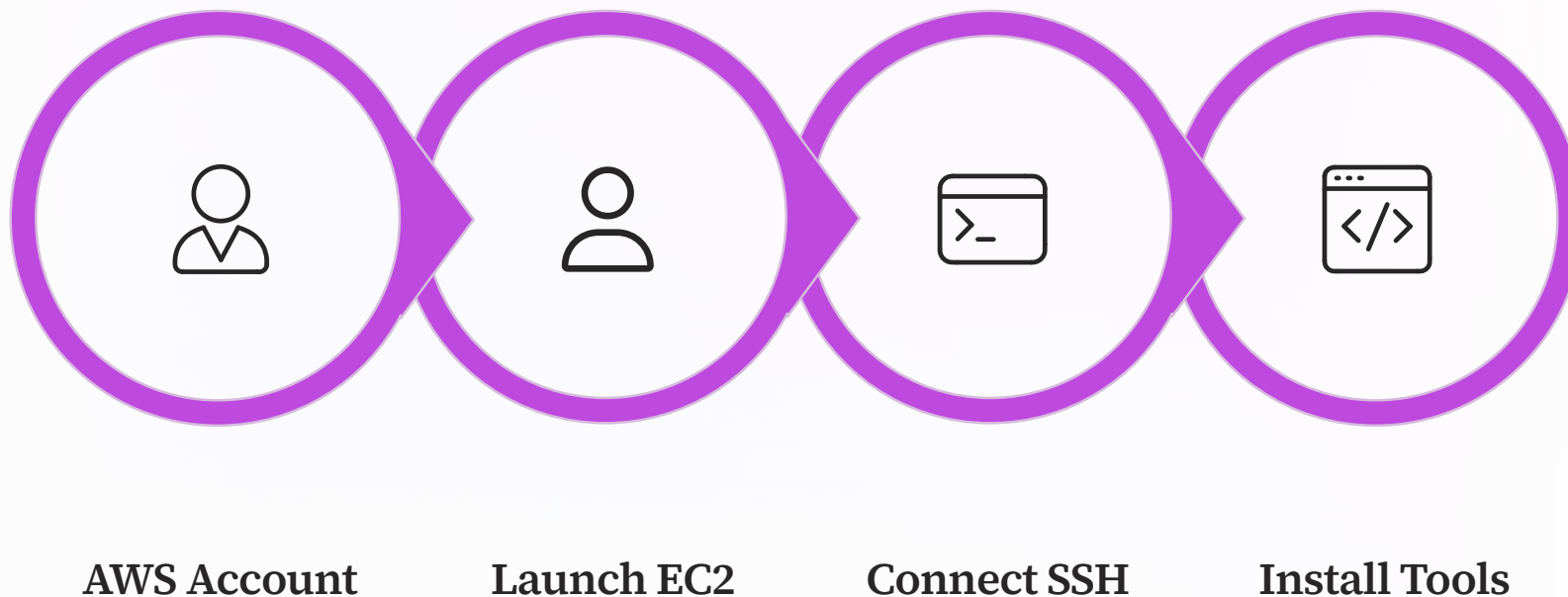
Instead of Googling for 30 minutes, describe your exact error to an AI. You'll get a targeted, step-by-step resolution in under a minute.

- ❏ Pro tip: Always include the **exact error message** and your OS in the prompt for the best results.

Common EC2 Issues AI Can Fix

- SSH: Permission denied (publickey)
- SSH: Connection timed out (security group)
- Instance won't start (storage/limit issues)
- Can't reach port 8080 (firewall rules)

Lab Overview: What You'll Build Today



By the end of Day 1, you'll have a live, accessible Linux server in the cloud — fully configured and ready for Jenkins installation on Day 2.

Step-by-Step: Launching Your EC2 Instance

1 Sign in to the AWS Console

Go to **aws.amazon.com** → Services → EC2 → Launch Instance

2 Choose an AMI

Select **Amazon Linux 2023** (Free Tier eligible)

3 Select Instance Type

Choose **t2.micro** — 1 vCPU, 1 GB RAM, Free Tier

4 Create a Key Pair

Name it `jenkins-key`, download the **.pem file**, and store it safely

5 Configure Security Group

Allow **SSH (port 22)** from your IP. Click **Launch Instance**.

Connecting to Your EC2 via SSH

01

Set Key Permissions (Mac/Linux)

```
chmod 400 jenkins-key.pem
```

02

Find Your Public IP

In the EC2 Console → Instances → copy the **Public IPv4 address**

03

SSH Into Your Instance

```
ssh -i jenkins-key.pem ec2-user@<your-public-ip>
```

04

Windows Users

Use **MobaXterm** or **PuTTY** with your .pem file converted to .ppk



If you see "**Permission denied (publickey)**" — check your .pem file path and that port 22 is open in your security group.

Installing Java & Git on Your EC2 Instance

Install Java 17 (Required for Jenkins)

```
sudo yum update -y
sudo yum install java-17-amazon-corretto -y
java -version
```

Install Git

```
sudo yum install git -y
git --version
```

Why These Tools?

Java — Jenkins runs on the Java Virtual Machine (JVM). It must be installed before Jenkins can start.

Git — Jenkins pulls your source code from Git. It's the version control system at the heart of every CI/CD pipeline.

 Both are prerequisites for Day 2 — Jenkins installation begins the moment you connect.



✓ DAY 1 WRAP-UP

Day 1 Expected Outcomes

✓ Cloud Concepts

You can explain IaaS, PaaS, SaaS, and all three deployment models

✓ AWS EC2 Running

A live EC2 instance is running in your AWS account

✓ SSH Connected

You've successfully logged into your server from your local machine

✓ Tools Installed

Java 17 and Git are installed — you're ready for Day 2

Coming Up on Day 2



Install & Configure Jenkins

Get Jenkins running on your EC2 and access the dashboard



Connect GitHub to Jenkins

Link your source code repository to trigger automated builds



Build Your First CI Pipeline

Write a `Jenkinsfile` and watch your first automated build run



Make sure your EC2 instance stays running overnight — or you'll need to reassign your public IP in the morning!